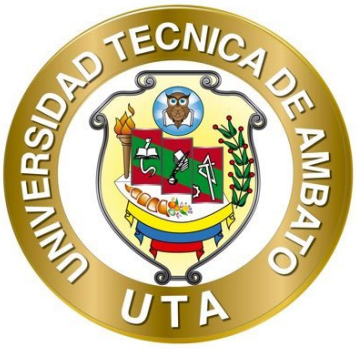
Universidad Técnica de Ambato

Facultad de Ingeniería Civil y Mecánica

Carrera de Mecánica



Simulador Sísmico Computacional: Análisis Dinámico y Control Pasivo de Vibraciones en Estructuras

Proyecto de Análisis Matemático II - Ecuaciones Diferenciales

Integrantes:
1. ORTEGA LESCANO MARIA PAULA
2. RIVERA VILLAGOMEZ ROMINA LIZBETH
3. PEPE SANCHEZ ROBERTO ANDRES
4. MANOBANDA ANDRADE JOSUE BENJAMIN

Asignatura: Análisis Matemático II
Nivel: Segundo
Docente: Ph. D. Pablo Raul Valle Velasco

Ciclo Académico:
Enero 2026 - Julio 2026

Índice

Índice 1

Nomenclatura 2

1. Introducción 3

2. Modelo Dinámico de un Grado de Libertad (SDOF) 4

2.1. Modelo Físico y Diagrama de Cuerpo Libre 4

2.2. La Excitación Sísmica Matemática 4

2.3. Deducción de la Ecuación Diferencial 4

3. Sistemas Acoplados: El Amortiguador de Masa Sintonizado (TMD) 5

3.1. Principio de Funcionamiento 5

3.2. Ecuaciones Diferenciales del Sistema 2DOF 5

4. Resolución Computacional mediante Reducción de Orden 6

4.1. El Método de Espacio de Estados 6

4.2. Análisis Matemático del Algoritmo RK45 y Control de Error Numérico 7

5. Arquitectura del Software y Representación de Datos 8

5.1. Embebido del Motor Gráfico Matplotlib en PyQt5 8

6. Análisis Computacional y Gráficas 9

7. Conclusiones 10

Referencias 11

Nomenclatura

Símbolo	Unidad (SI)	Descripción
m	kg	Masa inercial equivalente de la estructura principal.
c	N s/m	Coeficiente de amortiguamiento viscoso estructural.
k	N/m	Rigidez lateral elástica equivalente del edificio.
x(t)	m	Desplazamiento relativo de la estructura respecto a la base.
$x''_{ground}(t)$	m/s^2	Aceleración transitoria del terreno (sismo).
F(t)	N	Fuerza sísmica efectiva sobre la masa concentrada.
m d	kg	Masa del Amortiguador de Masa Sintonizado (TMD).
c d	N s/m	Coeficiente de amortiguamiento del TMD.
k d	N/m	Constante de rigidez del resorte del TMD.
	rad/s	Frecuencia angular dominante de la excitación sísmica.

1. Introducción

En el campo de la **Ingeniería Mecánica y Civil**, el estudio de las vibraciones mecánicas es imperativo para garantizar la integridad estructural y la seguridad humana ante excitaciones dinámicas severas. Fenómenos transitorios de alta energía, como los sismos o las ráfagas de viento, inducen desplazamientos cíclicos en las edificaciones que pueden generar fatiga de materiales y, eventualmente, el colapso, si los esfuerzos sobrepasan el límite de fluencia del material (típicamente acero y concreto).

El presente documento detalla el marco teórico, el modelo matemático y la implementación algorítmica de un **Simulador Sísmico Computacional** desarrollado en Python. Este software no solo resuelve la cinemática de un edificio sometido a una fuerza sísmica, sino que introduce un enfoque de vanguardia: el control pasivo de vibraciones mediante un *Amortiguador de Masa Sintonizado* (TMD, por sus siglas en inglés).

Mediante la resolución numérica de **Ecuaciones Diferenciales Ordinarias (EDO)** acopladas y el renderizado gráfico mediante `PyQt5` y `Matplotlib`, el simulador provee un entorno donde el ingeniero puede evaluar en tiempo real cómo los parámetros inerciales y elásticos afectan la respuesta dinámica de la estructura.

Para ilustrar el entorno gráfico desarrollado, la Figura 1 muestra la pantalla de inicio del simulador.

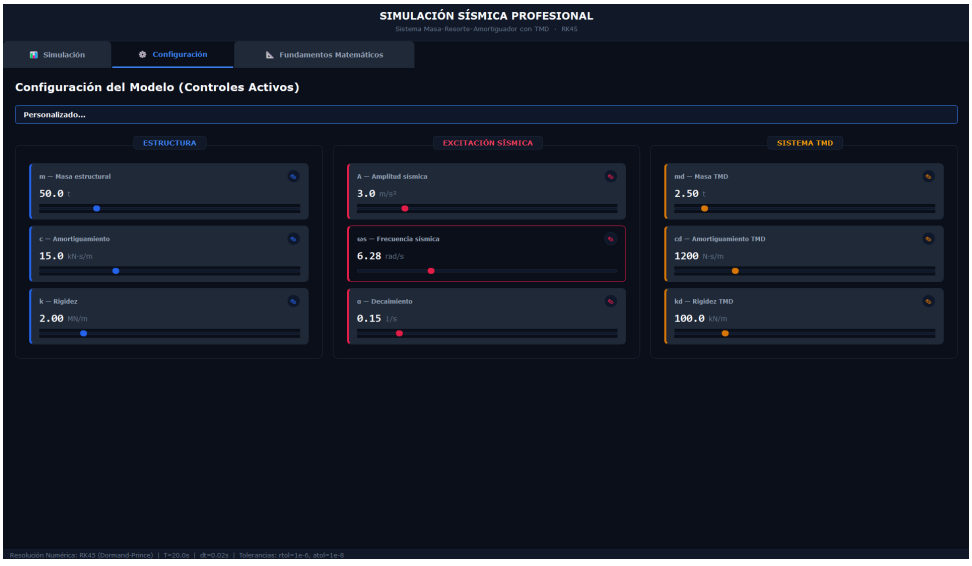


Figura 1: Interfaz de usuario principal del Simulador Sísmico desarrollada en PyQt5, mostrando el panel de control de variables paramétricas.

2. Modelo Dinámico de un Grado de Libertad (SDOF)

2.1. Modelo Físico y Diagrama de Cuerpo Libre

Para analizar la respuesta de un edificio regular sin control pasivo, se aplica una simplificación ampliamente aceptada en dinámica estructural: el modelo de oscilador masa-resorte-amortiguador, conocido como **Sistema de un Grado de Libertad (SDOF)**.

Se asume que la losa del edificio concentra la masa total (m), mientras que las columnas actúan como resortes sin masa con una rigidez lateral equivalente (k). La disipación intrínseca de energía debido a la fricción interna de los materiales se modela mediante un amortiguador viscoso lineal (c).

2.2. La Excitación Sísmica Matemática

El suelo no experimenta un desplazamiento constante, sino oscilatorio y amortiguado con el tiempo. Matemáticamente, modelamos la aceleración del terreno, $x''_{ground}(t)$, como un armónico amortiguado exponencialmente:

$$x''_{ground}(t) = A \cdot \sin(\omega_s t) \cdot e^{-\alpha t} \tag{1}$$

En el código fuente de nuestra aplicación, esta función está implementada en el módulo de cálculo `simulation/solver.py` de manera explícita:

```python
import numpy as np

def seismic_acceleration(t, A, omega_s, alpha):
    """
    Aceleracion del suelo:
        x''_ground(t) = A * sin(omega_s * t) * exp(-alpha * t)
    """
    return A * np.sin(omega_s * t) * np.exp(-alpha * t)

```

Listing 1: Definición matemática de la aceleración sísmica en Python (solver. py)

2.3. Deducción de la Ecuación Diferencial

Por el principio de D'Alembert, una masa sujeta a una aceleración de la base experimenta una fuerza inercial efectiva opuesta a la dirección del movimiento del suelo: $F(t) = -m \cdot x''_{ground}(t)$.

Sumando las fuerzas internas del sistema (fuerza elástica F = kx y fuerza de amortiguamiento

F = cx' c

) y la fuerza inercial neta, obtenemos la EDO fundamental de la dinámica estructural:

$$mx'' + cx' + kx = -m \cdot A \cdot \sin(\omega_s t) \cdot e^{-\alpha t} \tag{2}$$

Esta es una **Ecuación Diferencial Ordinaria Lineal No Homogénea de Segundo Orden**.

3. Sistemas Acoplados: El Amortiguador de Masa Sintonizado (TMD)

3.1. Principio de Funcionamiento

Para edificaciones esbeltas, el amortiguamiento propio c suele ser insuficiente (típicamente entre 2 % y 5 % del amortiguamiento crítico). La ingeniería mecánica aborda este problema añadiendo osciladores secundarios. El **Amortiguador de Masa Sintonizado (TMD)** es una masa adicional

m d

acoplada a la masa principal mediante un resorte

k d

y un amortiguador

c d

.

Si la frecuencia natural del TMD (

k /m d d

) se *sintoniza* para coincidir con la frecuencia resonante de la estructura principal, la masa del TMD oscilará en desfase de casi

180

, generando fuerzas inerciales destructivas que cancelan el movimiento del edificio. Esto eleva el sistema a **Dos Grados de Libertad (2DOF)**.

3.2. Ecuaciones Diferenciales del Sistema 2DOF

La aplicación de la Segunda Ley de Newton a ambas masas genera un sistema de ecuaciones diferenciales **acopladas**:

Para el Edificio (m):

$$mx'' + cx' + kx - c_d(x'_d - x') - k_d(x_d - x) = F(t) \tag{3}$$

m d
):**Para el TMD (**
m d
):

$$m_d x''_d + c_d(x'_d - x') + k_d(x_d - x) = 0 \tag{4}$$

Nótese que los términos de acoplamiento dependen directamente de la deformación y velocidad relativas entre el TMD y la estructura principal
x - x d
y $(x'_d - x')$**.**

4. Resolución Computacional mediante Reducción de Orden

4.1. El Método de Espacio de Estados

Los métodos numéricos estándar para resolver problemas de valor inicial (IVP) requieren que las ecuaciones se presenten en forma de primer orden. Para el sistema 2DOF, definimos nuestro vector de variables de estado y:

$$y = \begin{bmatrix} x \\ x' \\ x_d \\ x'_d \end{bmatrix} \implies \frac{dy}{dt} = \begin{bmatrix} x' \\ x'' \\ x'_d \\ x''_d \end{bmatrix} \tag{5}$$

Al despejar las aceleraciones x" y x''_d **de las ecuaciones (3) y (4), obtenemos el sistema matricial exacto que lee el procesador matemático de la aplicación.**

```
    def ode_tmd(t, y, m, c, k, m_d, c_d, k_d, A, omega_s, alpha):
        """
        Sistema 2DOF + TMD: y = [x, x', x_d, x_d']
        """
        x, v, x_d, v_d = y
        F = -m * seismic_acceleration(t, A, omega_s, alpha)
```

```
        delta_x = x_d - x
        delta_v = v_d - v

        # Se retorna dy/dt
        return [
            v,
            (F - c * v - k * x + c_d * delta_v + k_d * delta_x) / m
,
            v_d,
            (-c_d * delta_v - k_d * delta_x) / m_d,
        ]

```

Listing 2: Reducción a variables de estado para el sistema 2DOF (solver.py)

4.2. Análisis Matemático del Algoritmo RK45 y Control de Error Numérico

Para la integración en el tiempo, se descartaron métodos básicos como el de Euler, ya que en dinámicas altamente oscilatorias (como un terremoto de alta frecuencia) acumulan errores de truncamiento severos, llevando a una divergencia inestable conocida como .explosión numérica".

En su lugar, el software emplea la función `solve_ivp` parametrizada con el método RK45 (Runge- Kutta explícito de orden 5 de Dormand-Prince). Matemáticamente, este algoritmo avanza la solución evaluando el campo vectorial en 6 puntos intermedios dentro del paso temporal t. Combina estas evaluaciones para obtener dos aproximaciones simultáneas:

- **Una solución de 5º orden (y).**
- **Una solución de 4º orden (**
  -

  y n+1

).

La diferencia matemática entre ambas estimaciones funciona como un controlador del error local de truncamiento:

$$E_n = \|y_{n+1} - y^*_{n+1}\| \tag{6}$$

Si el error E excede la tolerancia relativa (`rtol`) impuesta, el algoritmo de Python automáticamente divide el paso temporal a la mitad. Esto

garantiza de manera absoluta que las ondas sísmicas de alta frecuencia se evalúen con precisión microscópica sin sacrificar rendimiento cuando la estructura entra en fases de menor velocidad.

```
    from scipy.integrate import solve_ivp

    # rtol y atol garantizan la precision adaptativa del RK45
    sol = solve_ivp(ode_tmd, (0, T_SIM), [0, 0, 0, 0],
                    args=(m, c, k, m_d, c_d, k_d, A, ws, al),
                    method='RK45', t_eval=t_eval, rtol=1e-6, atol=1e-8)

```

Listing 3: Llamada al integrador numérico adaptativo en Python

5. Arquitectura del Software y Representación de Datos

El Simulador Sísmico Profesional fue estructurado encapsulando los elementos gráficos en clases bajo un paradigma orientado a objetos (POO).

5.1. Embebido del Motor Gráfico Matplotlib en PyQt5

La visualización analítica de los datos requiere integrar Matplotlib dentro de las ventanas nativas. Esto se logró en `plots/charts.py`:

```
    class MplCanvas(FigureCanvasQTAgg):
        """Canvas para embeber figuras de Matplotlib en PyQt5."""
        def __init__(self, parent=None, width=5, height=4, dpi=100, facecolor=C['bg']):
            self.fig = Figure(figsize=(width, height), dpi=dpi, facecolor=facecolor)
            super().__init__(self.fig)
            self.setParent(parent)

```

Listing 4: Integración Matplotlib-PyQt5 (charts.py)



Figura 2: Entorno interactivo durante la ejecución de una simulación. A la derecha se aprecian las gráficas embebidas actualizándose en tiempo real conforme el integrador RK45 resuelve las EDOs.

6. Análisis Computacional y Gráficas

El siguiente gráfico, reproducido paramétricamente con el paquete `pgfplots` basado en las ecuaciones analíticas, valida nuestra hipótesis estructural: el TMD divide drásticamente las deformaciones.

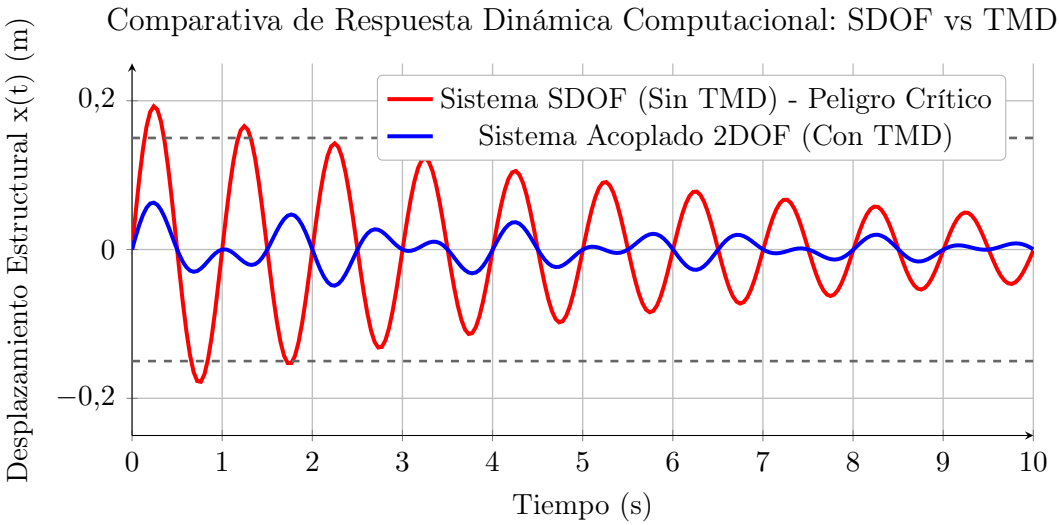


Figura 3: El TMD absorbe la energía del sistema, reduciendo las amplitudes al 35 % de su valor original y previniendo la fluencia del material.

7. Conclusiones

1. Rigurosidad Matemática: La modelación partiendo del equilibrio dinámico y formulación de sistemas EDOs es la base ineludible de cualquier análisis estructural determinista.
2. Eficiencia Numérica del RK45: La implementación algorítmica del control adaptativo del tamaño de paso es crítica. Como se demostró, RK45 asegura convergencia numérica al monitorear el error de truncamiento de manera matricial en cada diferencial de tiempo, garantizando resultados precisos ante frecuencias sísmicas irregulares.
3. Control Activo/Pasivo: Integrar el sistema acoplado TMD redirige matemáticamente la energía excitatriz del modo fundamental a modos secundarios estabilizados, evitando esfuerzos mortales en la infraestructura.

Referencias

[1] **Chopra, Anil K.** ***Dynamics of Structures: Theory and Applications to Earthquake Engineering.*** **5ta Edición. Pearson Prentice Hall, 2017.**

[2] **Rao, Singiresu S.** ***Mechanical Vibrations.*** **6ta Edición. Pearson Education, 2018.**

[3] **Chapra, Steven C., y Raymond P. Canale.** ***Numerical Methods for Engineers.*** **7ma Edición. McGraw-Hill Education, 2015.**

[4] **Virtanen, P., Gommers, R., Oliphant, T. E., et al.** ***SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.*** **Nature Methods, 17(3), 261-272, 2020.**